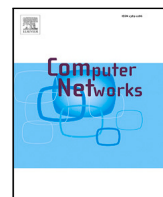




Contents lists available at ScienceDirect

Computer Networks

journal homepage: [www.elsevier.com/locate/comnet](http://www.elsevier.com/locate/comnet)



# FastTraffic: A lightweight method for encrypted traffic fast classification

Yuwei Xu <sup>a,b,c,\*</sup>, Jie Cao <sup>a,c</sup>, Kehui Song <sup>c,d</sup>, Qiao Xiang <sup>e</sup>, Guang Cheng <sup>a,b,c</sup>

<sup>a</sup> School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, China

<sup>b</sup> Purple Mountain Laboratories, Nanjing, 211111, China

<sup>c</sup> Jiangsu Province Engineering Research Center of Security for Ubiquitous Network, Nanjing, 211189, China

<sup>d</sup> College of Computer Science, NanKai University, Tianjin, 300071, China

<sup>e</sup> School of Informatics, Xiamen University, Xiamen, 361005, China

## ARTICLE INFO

### Keywords:

Encrypted traffic classification

Lightweight model

N-gram feature

Deep learning

## ABSTRACT

Nowadays, most Internet communications have adopted encrypted network access technology for privacy protection, so encrypted traffic classification (ETC) has become a crucial research point to support network management and ensure cyberspace security. Meanwhile, off-the-shelf deep learning (DL)-based approaches suffer from long preprocessing time, large input size, and a trade-off between model complexity and accuracy. There is a tough challenge to deploy them on mainstream network devices and achieve fast and accurate traffic classification. In this paper, we design FastTraffic, a lightweight DL-based method for ETC on low-configuration network devices. To speed up processing, we set an IP packet as the granularity of FastTraffic, truncate the informative parts in packets as inputs, and utilize a text-like packet tokenization method. For a lightweight and effective model, we propose an N-gram feature embedding method to represent structured and sequential features of packets and design a three-layer MLP to complete fast classification. We compare FastTraffic with eight state-of-the-art ETC methods on three public benchmark datasets. The experimental results show that FastTraffic obtains better classification performance than the other seven methods with only 0.43M model parameters. Besides, it can also achieve high throughput on low-configuration devices and consume a small amount of computing and storage resources. Therefore, FastTraffic is a lightweight ETC method capable of large-scale deployment on Internet devices.

## 1. Introduction

With the awakening of security awareness, Internet users realize the importance of privacy protection and gradually use encrypted network communication tools or protocols for network access. Studies from Firefox [1] reportedly counted that the percentage of U.S. users leveraging the SSL/TLS security protocol rose from 55% in 2017 to 93% in 2022. In addition, Virtual Private Network (VPN) and The Onion Router (Tor) are also widely employed by users to ensure the anonymity of network communications. While improving communication privacy, users also hope to obtain a high-quality network experience. Internet service providers (ISPs) are responsible for charging users based on the quality of experience (QoE) provided. However, the introduction of encryption technology protects the confidentiality of transmitted data, while the adoption of obfuscation technology increases the difficulty of traffic identification. It is difficult for ISPs to identify various types of encrypted traffic on network devices. In this context, the encrypted traffic classification (ETC) technique is envisioned as the critical enabler of

guaranteeing the quality of service (QoS), QoE, and network security provision [2].

As aforementioned, encrypted network environments pose significant challenges to traditional traffic classification methods, including port-based traffic identification methods and deep packet inspection (DPI) technology. Port-based identification methods utilize the correspondence between ports and protocols for traffic classification but become ineffective due to the widespread use of port forwarding and dynamic port negotiation techniques. DPI identifies specific protocol and application data by analyzing the entire IP packet. Since it cannot parse encrypted packet payloads, DPI cannot achieve encrypted traffic classification [3].

So far, researchers have proposed many methods to classify encrypted traffic by leveraging artificial intelligence technologies. There are two categories: machine learning (ML)-based and deep learning (DL)-based. ML-based methods usually use statistical patterns of traffic to construct traffic features, like packet path signature [4], packet

\* Corresponding author at: School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, China.

E-mail addresses: [xuyw@seu.edu.cn](mailto:xuyw@seu.edu.cn) (Y. Xu), [cao.jie@seu.edu.cn](mailto:cao.jie@seu.edu.cn) (J. Cao), [songkehui@nankai.edu.cn](mailto:songkehui@nankai.edu.cn) (K. Song), [qiaoxiang@xmu.edu.cn](mailto:qiaoxiang@xmu.edu.cn) (Q. Xiang), [chengguang@seu.edu.cn](mailto:chengguang@seu.edu.cn) (G. Cheng).

<https://doi.org/10.1016/j.comnet.2023.109965>

Received 6 March 2023; Received in revised form 1 June 2023; Accepted 2 August 2023

Available online 6 August 2023

1389-1286/© 2023 Elsevier B.V. All rights reserved.

arrival time and size [5], and cumulative packet sizes (CUMUL) [6], the best-performing classifier is then chosen to classify them. The main research topic and also a dilemma of ML-based methods is that they must accomplish feature engineering to select optimal features for each specific encrypted traffic protocol or tool. This work is highly dependent on domain expert knowledge, and the selected features are commonly less generalization [7]. However, in real network scenarios, it is time-consuming and prohibitive to train and implement different ML-based ETC methods for various encrypted traffic generated by different tools or protocols.

Given the powerful representation capability of deep neural networks, researchers put forward numerous DL-based methods for classifying encrypted traffic [8]. The merit of DL is that the model can directly input raw traffic data and automatically perform feature extraction and representation during the supervised training process, thus eliminating the requirement for feature engineering and expert knowledge in ML. The DL-based method is naturally suitable for early ETC scenarios, which signifies a clear difference from ML-based approaches that depend on complete flow or session. However, there are two shortcomings of existing DL-based methods. Firstly, they did not explore the input size of the classification model by analyzing packet format. In previous work, the researchers often set the input size according to the maximum transmission unit (MTU). The unreasonable values increase preprocessing time overhead and cause a waste of resources on devices. Secondly, many DL-based methods pursued superior classification performance at the expense of operational efficiency. To explore the feature of encrypted traffic, the model structure adopted by ETC methods shifted from a single Convolutional Neural Network (CNN) [9] or Recurrent Neural Network (RNN) [10] to a stack of CNNs and RNNs [11,12]. Recently, Transformer [13] has also been introduced into the ETC field owing to its outstanding performance in NLP and CV. Although these complex models can achieve excellent classification performance on high-configuration workstations, mainstream network devices on the Internet cannot provide the same hardware resources, especially high-performance graphics cards (GPUs). With the research trend of adopting complex network models, DL-based methods occupy more and more hardware resources. Once deployed on network devices with limited resources, they will affect regular network functions, which goes against the original intention of guaranteeing QoS and QoE [14].

To deploy on mainstream network devices on the Internet and achieve fast and accurate traffic classification, we propose FastTraffic, a lightweight ETC method, including a traffic preprocessing method and a classification model. The main contributions of our work lie in the following three aspects.

- Firstly, we propose a traffic preprocessing method. A suitable and reasonable packet truncation length, 50 bytes, is determined via protocol analysis and experiments. Besides, we perform tokenization with one byte of hexadecimal characters as one word to reduce the input size. With this method, we effectively preprocess the encrypted traffic and thus lay the groundwork for subsequent data representation and reduce the time overhead from the outset.
- Secondly, we design a lightweight classification model. In the packet embedding part, we propose N-gram feature embedding to extract the structural and sequential information in the raw bytes, aiming to provide a better packet representation. In the classification network part, we use a three-layer MLP to reduce the number of model parameters. In addition, we analyze the complexity of the FastTraffic classification model at the theoretical dimension and prove that its model complexity is lower than state-of-the-art methods.
- Thirdly, to assess the effectiveness of FastTraffic, we conduct comprehensive experiments and reproduce eight contrastive ETC methods, including two lightweight methods. We introduce four ETC tasks from three benchmark datasets and test these methods

on high-configuration and low-configuration devices. The results show that FastTraffic obtains high classification performance and excellent operational efficiency. It is suitable for large-scale deployment on mainstream network devices, achieving fast and accurate traffic classification.

The rest of this paper is organized as follows. In Section 2, we introduce previous work and show the motivation of FastTraffic. In Section 3, three public datasets are described first, and then the comprehensive workflow of the data preprocessing method is elaborated. The design of our classification model and its complexity analysis are arranged in Section 4. We introduce the experimental setup in Section 5 and verify the classification performance and operational efficiency of FastTraffic through comparative experiments in Section 6. Finally, we conclude our work in Section 7.

## 2. Related work

In this section, we review the existing works in ETC and underline the motivation of our work.

### 2.1. ML-based method

ML-based methods are proposed to tackle the problem that traditional traffic classification methods cannot cope with dynamic port negotiation and encrypted payload. Feature selection and optimization play a vital role in ML-based methods, thus researchers proposed lots of features based on the statistical pattern of the traffic, and selected the appropriate ML classifiers, such as support vector machine, random forest, etc., to classify them.

In [17], the authors proposed to use only the uplink's packet timing information to measure VPN traffic. The cumulative packet sizes feature (CUMUL) was first proposed in [6] to produce the Tor website fingerprinting and was widely used by subsequent researchers. NetScope [24] was proposed to infer both Android and iOS device users' activities based on traffic behaviors transfer rates, packet exchanges, etc. In [18], the authors proposed a video download feature called Bit-Per-Peak, which is the total number of bits in a peak. Shen et al. [15] combined the CUMUL with the packet position on the uplink-dominant stage to construct a novel fine-grained website fingerprinting. ETC-PS [4] constructed the traffic path between the encrypted communication client and the server via the session packet length sequence, and extracted path signature features based on this path.

Table 1 provides a summary of ML-based methods, and their commonality is that their granularity is flow or session because achieving the pattern of communication and extracting statistical features need to observe the entire flow or session i.e. it is infeasible to implement early ETC, which is time consuming. At the same time, ML-based methods were all employed at the foundation of feature engineering, which relied on expert knowledge. Nevertheless, researchers' feature selection efforts focused on specific encryption tools or protocols, and these selected feature sets lacked generalization ability. Hence, ML-based methods cannot rapidly select uniform features for classifying encrypted traffic and are not suitable for real network scenarios where traffic types and sources are variable.

### 2.2. DL-based method

In NLP and CV, DL-based methods have grown significantly, and many classical neural network models from them have been applied to the ETC field. DL-based methods did not require manually selecting features. Instead, they directly feed the model with the raw traffic and generated features automatically and depending on the input granularity can be divided into three levels, including session, flow, and packet, as shown in Table 1.

**Table 1**

A summary of existing encrypted traffic classification methods.

| Ref ID.     | Type | Classifier       | Granularity | Feature                        | Early ETC | Input size | Used dataset            | Lightweight |
|-------------|------|------------------|-------------|--------------------------------|-----------|------------|-------------------------|-------------|
| [6]         | ML   | SVM              | Session     | CUMUL                          | ✗         | –          | Self-collect Dataset    | ✗           |
| [15]        | ML   | RF               | Session     | CUMUL                          | ✗         | –          | JD, YH dataset          | ✗           |
| [4]         | ML   | RF               | Session     | Packet path signature          | ✗         | –          | ISCX-VPN, Tor, etc.     | ✗           |
| [16]        | ML   | SVM              | Flow        | Statistical TCP flow features  | ✗         | –          | Malicious traffic       | ✓           |
| [17]        | ML   | KNN+Naive Bayes  | Flow        | Packet timing information      | ✗         | –          | VPN, Tor (self-collect) | ✗           |
| [18]        | ML   | SVM, NN          | Flow        | Total number of bits in a peak | ✗         | –          | YouTube (self-collect)  | ✗           |
| [19]        | DL   | CNN              | Session     | Raw session bytes              | ✓         | 28 × 28    | ISCX-VPN                | ✗           |
| [20]        | DL   | BiLSTM+Attention | Flow        | Raw flow bytes                 | ✓         | 10 × 1500  | ISCX-VPN                | ✗           |
| [21]        | DL   | Transformer      | Flow        | Raw flow bytes                 | ✓         | 3 × 786    | CSTNET, etc.            | ✓           |
| [11]        | DL   | CNN+LSTM         | Flow        | Raw flow bytes                 | ✓         | 15 × 1500  | ISCX-Tor, USTC-TFC      | ✗           |
| [9]         | DL   | CNN              | Packet      | Raw packet bytes               | ✗         | 1500       | ISCX-VPN, Tor           | ✗           |
| [22]        | DL   | MLP, SAE, CNN    | Packet      | Raw packet bytes               | ✗         | 1480       | ISCX-VPN                | ✓           |
| [23]        | DL   | Attention+CNN    | Packet      | Raw packet bytes               | ✓         | 50         | ISCX-VPN, WIDE          | ✗           |
| [13]        | DL   | BERT             | Packet      | Raw packet bytes               | ✓         | 256        | ISCX-VPN, Tor, etc.     | ✗           |
| FastTraffic | DL   | MLP              | Packet      | Raw packet bytes               | ✓         | 50         | ISCX-VPN, Tor, USTC-TFC | ✓           |

Researchers have followed the iterations of models in artificial intelligence field and have also continued to propose various DL classification models. The most important advantage of the DL model is the automated feature representation of the traffic raw data. First of all, CNN as an effective structure for automatic extraction of features is also applied to ETC, for example, DeepPacket [9] method is designed based on the 1DCNN model. While RNN has advantages in processing time-series data, researchers have also proposed RNN-based methods [10] for ETC. Secondly, individual CNN or RNN cannot fully and comprehensively represent the raw flow or packet, so in [11,20], the authors proposed to stack CNN with RNN to further improve the model's ability to represent traffic. Finally, the Transformer structure, which is centered on the Self Attention mechanism (SA) and Multi-head Attention mechanism (MHA), has a larger perceptual field than CNN and stronger time-series data representation ability than RNN and has obtained great success in NLP and CV fields in recent years. Thus, Transformer has also been introduced by researchers in the ETC field. Lin et al. [13] proposed ET-BERT, which is an excellent method. They constructed this novel pre-training method for ETC inspired by Transformer design in [25] and fine-tuned it for different downstream tasks on this basis. The accuracy of ET-BERT for seven ETC tasks can reach 97% and above.

In addition to the conventional methods listed in Table 1, researchers have proposed many emerging ETC methods. In [26], the authors designed a framework for multi-task learning based on 1DCNN to improve the generalization ability and resource utilization of DL models. The idea of multi-task learning was also adopted in [27]. Meanwhile, researchers proposed DL-based multimodal ETC methods to improve classification performance. In [28], the authors used packet length sequences and packet load bytes as the input of two modalities fed in parallel to the model to learn the joint characterization. In [29,30], CNN and GRU were employed to extract features from the L4 payload and header field of packets, respectively. In [31], the authors optimize Distiller [30] for deployment on resource-constrained devices by pruning, quantification, and knowledge distillation from the perspective of eXplainable Artificial Intelligence (XAI).

As can be observed, DL-based approaches accompanying the development of artificial intelligence, tend to create sophisticated model structures to classify encrypted traffic, which does not comply with online traffic classification and cannot ensure QoS. To sum up, the first problem of DL-based methods was that the researchers did not conduct a more reasonable exploration of the input format construction and input size selection of the ETC model, leading to blindly consistent processing of inputs with large sizes [9]. It increases the time overhead of preprocessing and also affects the model's input layer design. The second one was that the existing works, like [11,13,30], tended to introduce complex and heavyweight neural network in pursuit of high accuracy, which dramatically reduced the efficiency of the ETC method and does not meet the online classification and real deployment requirements of guaranteeing.

### 2.3. Motivation

Given deployment on real-world network devices, researchers have proposed several lightweight ETC methods such as MATEC [22] and Datanet [21]. The common thread of these studies is to reduce the input data and simplify the model structure. Although MATEC aims to be lightweight, it retains a complex network structure in the model to maintain high classification performance, resulting in many model parameters and much resource usage. Datanet adopts a four-layer MLP to construct its classification model. It does meet the requirement of lightweight, but its poor data expression capability has caused a decrease in accuracy.

To address the shortcomings of previous work, we propose FastTraffic intending to deploy on mainstream network devices and achieve fast and accurate traffic classification. The novelty of FastTraffic lies in two points. Firstly, we put forward a traffic preprocessing method. A suitable and reasonable packet truncation length, 50 bytes, is determined via protocol analysis and experiments. Besides, we perform tokenization with one byte of hexadecimal characters as one word to reduce the input size. Secondly, we design a lightweight classification model. In the packet embedding part, we propose N-gram feature embedding to extract the structural and sequential information in the raw bytes, aiming to provide a better packet representation. In the classification network part, we use a three-layer MLP to reduce the number of model parameters. Experimental results show that FastTraffic has significant advantages over state-of-the-art ETC methods and is suitable for deployment on network devices with limited resources.

### 3. Data preprocessing of FastTraffic

In this section, we first introduce three well-known public datasets covering the typical application fields of network encrypted traffic and then elaborate on the workflow of our preprocessing method.

#### 3.1. Dataset

In order to guarantee the general applicability of our method, three most cited public datasets in the field of network traffic research are selected, including *ISCX-VPN*, *ISCX-Tor* and *USTC-TFC*. In the following study, we will not only determine the truncation length by analyzing the packets of these datasets but also evaluate the performance of FastTraffic on their classification tasks.

- (1) The *ISCX-VPN* dataset is provided by Canadian Institute for Cybersecurity (CIC) [32]. The network traffic is captured using Wireshark and stored in PCAP files. The whole dataset contains 193GB real-world traffic data. All samples are set with three categories of labels, namely VPN and non-VPN, 11 types of network service, and 15 types of network application.

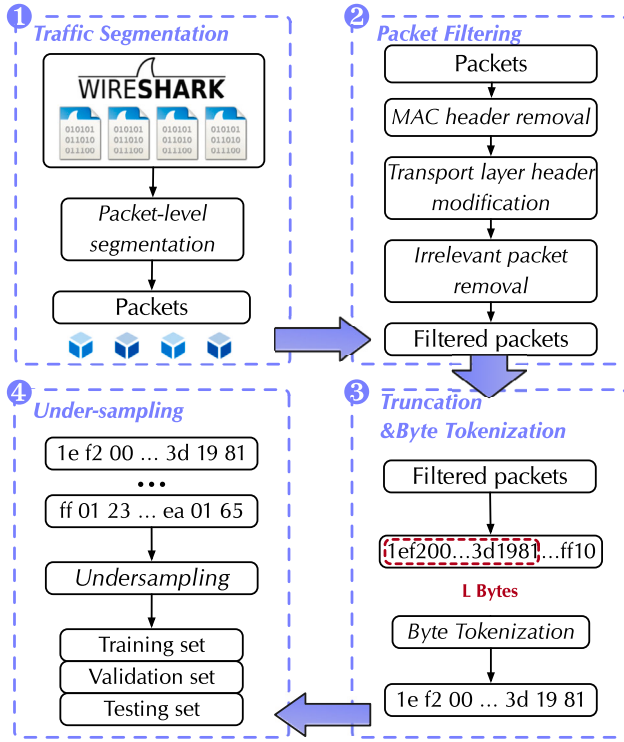


Fig. 1. The workflow of FastTraffic's preprocessing.

- (2) The *ISCX-Tor* dataset is also provided by CIC [33], which contains 22.8GB of Tor network traffic data. Since Tor aims at anonymous communication, it adopts traffic encryption and obfuscation technologies when transmitting application data. Therefore, its traffic identification has been a difficult and hot topic in ETC research. In *ISCX-Tor*, all samples contain two categories of labels, namely Tor and non-Tor, and 11 types of Tor application.
- (3) The *USTC-TFC* dataset is published on GitHub by Wei Wang [34]. It is widely cited in intrusion detection studies because it provides the traffic of 10 well-known malware. The *USTC-TFC* contains 3.7GB of data, and all samples are set with two categories of labels, namely goodware and malware, and 20 types of software.

### 3.2. Data preprocessing workflow

Since the task of data preprocessing is to provide raw data of uniform size to the classification models as input, it is an important issue worthy of discussion in DL-based network traffic classification. As shown in Fig. 1, the process of data preprocessing includes four steps, traffic segmentation, packet filtering, truncation & byte tokenization, and under-sampling.

#### 3.2.1. Traffic segmentation

In FastTraffic, we set an IP packet as the classification granularity to improve the efficiency of the classification model. It is worth noting that although a finer-grained segmentation is adopted, the labels of each packet  $p$  should be set according to the flow  $f$  to which it belongs. Therefore, we first extract the flows from the original PCAP file, still based on the common 5-tuple  $\{srcIP, srcPort, dstIP, dstPort, Protocol\}$ , and then segment the packets from each flow. According to the above strategy, we can quickly generate labeled packets from all the PCAP files by leveraging SplitCap.

#### 3.2.2. Packet filtering

Due to the complexity of the network environment, traffic datasets often contain a portion of packets that are irrelevant to the classification tasks and need to be filtered out. In addition, before the packets are fed into the classification model as input data, some fields need to be specially processed to avoid adverse effects on the model. As shown in Fig. 1, packet filtering consists of four operations.

- (1) Some packets are necessary for maintaining data communication over the Internet but are not related to the applications and services labeled in classification tasks. Based on this consideration, we filter out DNS and ICMP packets after traffic segmentation.
- (2) Since the MAC layer only provides information about the underlying communication of the local network, it does not contain global information for Internet transmission. This limitation of data will affect the training result of the classification model. Therefore, we remove the header and tail of the Ethernet frame and reserve the IP packet as the input of FastTraffic.
- (3) Due to limited conditions, experimenters usually can only deploy a few devices to connect to specified services during traffic collection, so the number of addresses included in the dataset cannot cover the address range of the entire Internet. To prevent classification tricks caused by high-frequency addresses, we mask the source and destination addresses in IP packets by 0.0.0.0. Besides, we set all source ports to 0 to avoid the experimental biases demonstrated in [35].
- (4) As the data transmission requirements are different, the network traffic flows in the three datasets adopt both the TCP and UDP protocols. The length of the UDP header is shorter than that of the TCP header. To provide uniform-length data input to the model, we pad 12 bytes of 0x00 after the UDP header. Besides, some control packets used to maintain TCP connections are removed because they do not contain the payload of upper-layer applications.

#### 3.2.3. Truncation & byte tokenization

The length of IP packets is variable, but the classification model based on deep learning requires input data of uniform size. Therefore, truncating long packets and padding short packets is an essential step. As discussed in Section 2, padding all packets to the MTU size preserves all original data but increases the processing time overhead of the classification model. To build a lightweight model under the premise of ensuring classification performance, it is an important issue to choose a reasonable truncation length  $\mathcal{L}$  for IP packets. For the optimal value of  $\mathcal{L}$ , we propose an interval-based searching strategy. By analyzing the data in three datasets, we set a byte interval for  $\mathcal{L}$  according to the length distribution of IP packets and the format of related protocols. In the training phase, the classification model only needs to make hyperparameter tuning within this interval, thus reducing the number of parameters and shortening the processing time.

Fig. 2 shows the distributions of IP packet length. In the three datasets, 41%, 51%, and 76% of the packets are less than 100 bytes in length, respectively. If  $\mathcal{L}$  is set to exceed 100 bytes, many packets need to be padded to achieve a uniform input size. In addition, we analyze the encapsulation structure of IP packets. Compared with random data payloads, the headers at each layer of the network protocol stack can provide more meaningful information for ETC. Through the statistics of the network traffic, we find that most of the packets include TLS headers in addition to IP headers and TCP/UDP headers. In the papers [23,36], the authors have obtained good classification results by using the first 50 bytes of the IP packets, proving that the header information of each layer protocol is effective for ETC. Based on the above analysis, we set the interval of  $\mathcal{L}$  to [50, 100] to discard the payload of long packets and retain all types of header information in IP packets. In Section 5, the value of  $\mathcal{L}$  will be determined within this interval by random hyperparameter tuning.



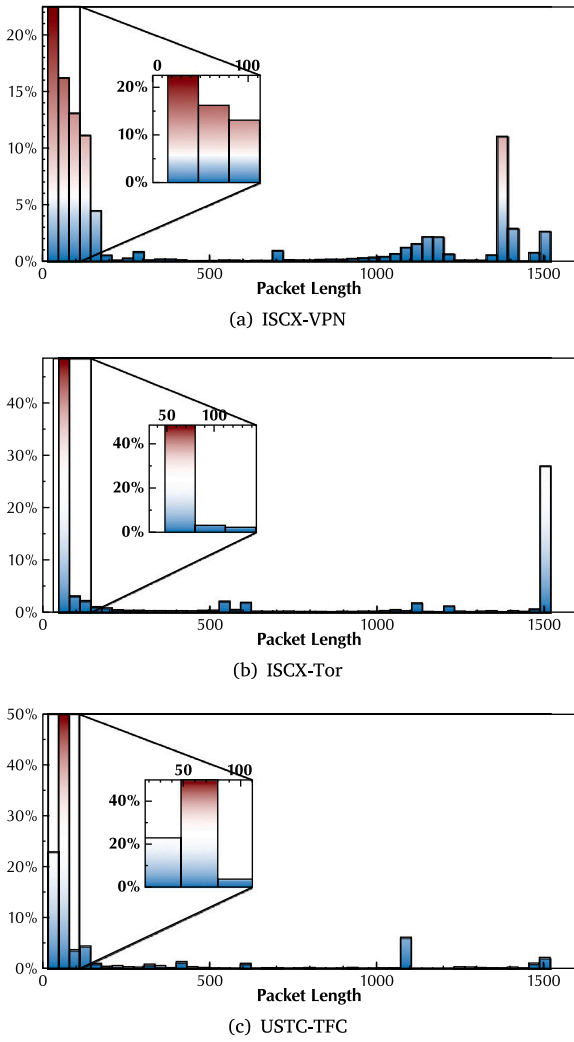


Fig. 2. Distribution of IP packet lengths in three public encrypted traffic datasets.

Drawing on the idea of NLP research, we handle the original data of each IP packet as a piece of text in FastTraffic. As shown in Fig. 1, the truncated IP packet undergoes treatment as a fixed-length sequence of bytes, which is then disassembled into tokens. Each byte is viewed as a token, symbolized by two hexadecimal characters. After the above operation, we convert each IP data packet into a token sequence whose length is  $\mathcal{L}$ .

### 3.2.4. Under-sampling

Data imbalance is a common issue that affects the performance of ETC. FastTraffic is a packet-level approach that deals with each IP packet as one sample. To prevent the high-frequency samples from attracting the attention of the classification model and thereby reducing the accuracy, we randomly under-sample the three datasets. According to the analysis in Section 3.1, we require at most 50,000 IP packets can be sampled for each category. If the number is less than 50,000, we use all packets of this category as samples.

## 4. FastTraffic classification model

This section first demonstrates the classification model details of our proposed FastTraffic and then analyze and compare the computational complexity with other methods' model theoretically.

Table 2

Notation used in FastTraffic.

|                  | Notation                | Definition                                                                       |
|------------------|-------------------------|----------------------------------------------------------------------------------|
| Model related    | $p_i$                   | The $i$ th IP packet in the flow.                                                |
|                  | $t_i$                   | The $i$ th byte in the IP packet.                                                |
|                  | $y_i$                   | True label of $p_i$ .                                                            |
|                  | $\mathcal{L}$           | Truncation length, retaining the first $\mathcal{L}$ bytes of each packet.       |
|                  | $D$                     | Embedding dimension.                                                             |
|                  | $N$                     | N-gram slide window size.                                                        |
|                  | $U^{(N)}$               | Trainable embedding vector for N-gram.                                           |
|                  | $k_i$                   | Index of $t_i$ in the uni-gram vocabulary.                                       |
|                  | $k_{i,i+1}$             | Index of $t_i t_{i+1}$ in the 2-gram vocabulary.                                 |
|                  | $k_{i-1,i,i+1}$         | Index of $t_{i-1} t_i t_{i+1}$ in the 3-gram vocabulary.                         |
|                  | $M$                     | Hash bucket size. It is the 2-gram and 3-gram vocabulary size, and set to 5000.  |
|                  | $\alpha, \beta, \gamma$ | Random large prime numbers.                                                      |
| Training related | $p_i^{(N\text{-gram})}$ | N-gram sequence for $p_i$ .                                                      |
|                  | $E_{p_i}$               | Concatenated vector for uni-gram, 2-gram, and 3-gram embedding.                  |
|                  | $X_{p_i}$               | Packet representation obtained by N-gram feature embedding for $p_i$ .           |
|                  | $\mathcal{H}$           | Hidden layer size of three-layer MLP.                                            |
|                  | $\mathbf{W}$            | Trainable weight.                                                                |
|                  | $\mathbf{b}$            | Trainable bias.                                                                  |
|                  | $\mathcal{R}$           | Dropout rate.                                                                    |
|                  | $ B $                   | Batch size.                                                                      |
|                  | $\mathcal{Y}$           | Class set for the ETC task.                                                      |
|                  | $\hat{y}_{p_i}$         | Predicted probability vector that $p_i$ belongs to each class in $\mathcal{Y}$ . |
|                  | $\hat{y}_{p_i}^c$       | $c$ th element of $\hat{y}_{p_i}$ .                                              |
|                  | $y_{ic}$                | The ground truth label.                                                          |

### 4.1. Model architecture

As shown in Fig. 3, the FastTraffic classification model has two components, including the packet embedding part and the classification network part. Owing to the analogy between the traffic bytes and text paragraph, the packet embedding part leverage the N-gram feature embedding aiming to learn the structural and sequential information within bytes and extracts the packet representation from the raw bytes. To achieve the lightweight goal, we propose a classification network based on a three-layer MLP to classify the packet representation. Each component will be explained in more detail below. Table 2 presents a summary of the mathematical notations that have been used in this paper, thus making it easier for readers to follow the content.

#### 4.1.1. Packet embedding

The purpose of embedding is to transform raw packet bytes into a vector, and some embedding design have been proposed in existing works.

First of all, an input traffic packet can be denoted as  $p_i = \{(t_1, t_2, \dots, t_{\mathcal{L}}), y_i\}$ ,  $y_{p_i}$  is the true label of this packet and  $t_i$  is each byte in the packet and defined as follows:

$$0x00 \leq t_i \leq 0xff$$

which can be obtained after preprocessing. This byte can be exactly equivalent to one word in a text paragraph, so we construct a vocabulary of size 256 and perform one-hot encoding for each  $t_i$ . For one-hot sparse representation, all byte representation vectors are orthogonal and cannot convey the correlation between bytes, so trainable embedding layers are introduced in FastTraffic to perform low-dimensional representation of bytes. For each  $p_i$ , the uni-gram embedding process can be expressed as Eq. (1), where  $p_i \in \mathbb{R}^{\mathcal{L}}$ ,  $\mathbf{U}^{(1)}$  is a trainable embedding vector, and  $E^{(1)} \in \mathbb{R}^{\mathcal{L} \times D}$ .  $D$  is the embedding dimension, which is an adjustable hyperparameter and will be determined in Section 5.

$$E^{(1)} = p_i \mathbf{U}^{(1)} \quad (1)$$

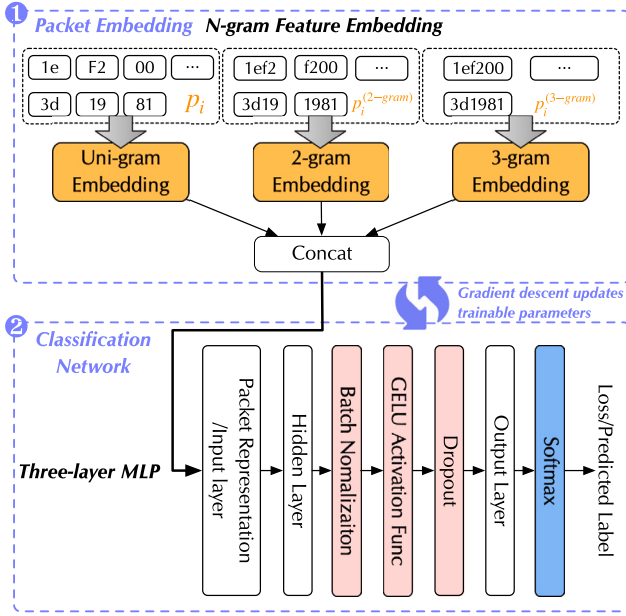


Fig. 3. The classification model of FastTraffic.

It is insufficient to utilize the uni-gram embedding, because it just offer the byte information and not the structural and sequential information inherent in the byte sequence. Therefore, the N-gram feature embedding is introduced to enrich the learnable packet information. N-gram itself is a feature embedding method in the NLP domain [37] and is a classical statistical language model that operates a sliding window of size  $N$  on the content of a sequence according to different token lengths. In the scenario of ETC, we introduce N-gram feature embedding for the following two reasons:

- (1) The byte sequence of an IP packet has a more fixed and rigorous structure than a text sequence. One, two, three, or four consecutive bytes are as fields and provide practical support for network traffic transmission. For instance, as shown in Fig. 4, the first three bytes of the TLS header specify the data type of the TLS payload and the TLS protocol version. In the packet header, only four fields are four bytes, thus avoiding excessive storage and computation overhead, we only design sliding windows with sizes  $N = 2$  and  $N = 3$  to extract the packet structural information.
- (2) In IP packets, there is a clear relative order between each byte position, so introducing this order can help us achieve a better representation of the packet. However, Uni-gram embedding is invariant to a byte sequence, and introducing this order via a well-known self-attention mechanism is computationally expensive. Therefore, N-gram feature embedding with sliding windows, whose stride is 1 byte, can capture the byte sequential information, as shown in Fig. 4.

Therefore, we use 2-gram (2-byte) and 3-gram (3-byte) features to construct two new token sequences, which can be denoted as Eq. (2)–(3). The  $p_i^{(2-gram)}$  and  $p_i^{(3-gram)}$  will be represented by two independent embedding vectors. The two low-dimensional vectors obtained after embedding will be concatenated together with  $E^{(1)}$ , and the mean value of the  $\mathcal{L}$  bytes will be calculated for obtaining the final representation of the input packet. This process is shown in Eqs. (4)–(6).  $U^{(2)}$  and  $U^{(3)}$  are trainable embedding vectors, and the concatenated vector is denoted as  $E_{p_i} \in \mathbb{R}^{\mathcal{L} \times 3D}$ . In Eq. (7), the  $e_{r,c}$  represents the element in row  $r$  and column  $c$  of the  $E_{p_i}$ , and the final representation of the input packet can be denoted as  $X_{p_i} \in \mathbb{R}^{3D}$ .

$$p_i^{(2-gram)} = \{t_1 t_2, t_2 t_3, \dots, t_{\mathcal{L}-1} t_{\mathcal{L}}\} \quad (2)$$

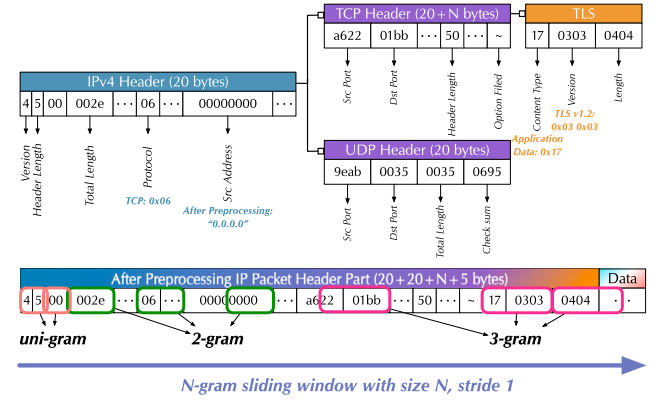


Fig. 4. Packet structure and N-gram construction.

$$p_i^{(3-gram)} = \{t_1 t_2 t_3, t_2 t_3 t_4, \dots, t_{\mathcal{L}-2} t_{\mathcal{L}-1} t_{\mathcal{L}}\} \quad (3)$$

$$E^{(2)} = p_i^{(2-gram)} U^{(2)} \quad (4)$$

$$E^{(3)} = p_i^{(3-gram)} U^{(3)} \quad (5)$$

$$E_{p_i} = [E^{(1)} \quad E^{(2)} \quad E^{(3)}] \quad (6)$$

$$X_{p_i} = \text{Mean}(E_{p_i}) = \left[ \frac{1}{\mathcal{L}} \sum_{r=1}^{\mathcal{L}} e_{r,1} \dots \frac{1}{\mathcal{L}} \sum_{r=1}^{\mathcal{L}} e_{r,3D} \right] \quad (7)$$

Towards 2-gram and 3-gram, we also need to build the corresponding vocabulary to complete the embedding. The scale of the vocabulary for 2-gram is  $256^2$ , and 3-gram reaches  $256^3$ . Consequently, building the complete vocabulary for 2-gram and 3-gram will incur huge computational and storage overhead, and not all 2-gram and 3-gram byte's combination occur in the byte sequence at high frequencies. Thus, a simple hash bucket trick is proposed to compress the vocabulary size based on [38] and improve the running speed. The formulas of the hash bucket are as follows:

$$k_{i,i+1} = \text{hash}_1(k_i, k_{i+1}) = (\alpha k_i + \beta k_{i+1}) \bmod M \quad (8)$$

$$k_{i-1,i,i+1} = \text{hash}_2(k_{i-1}, k_i, k_{i+1}) \\ = (\alpha k_{i-1} + \beta k_i + \gamma k_{i+1}) \bmod M \quad (9)$$

Where  $k_i$  is the index of  $t_i$  in the uni-gram vocabulary and  $M$  is the hash bucket size.  $\alpha$ ,  $\beta$ , and  $\gamma$  is a large prime numbers that we randomly choose. With this hash mapping, we establish the relationship between uni-gram to 2-gram and 3-gram token sequences and obtain the corresponding vocabulary to support the 2-gram and 3-gram embedding. As a result, we set the size of the 2-gram and 3-gram  $M$  to 5000 by experimental analysis. Significantly, benefiting from the fact that  $\mathcal{L}$  is limited to the interval [50, 100] in Section 3.2, the occurrence of low-frequency 2-grams and 3-grams can be effectively alleviated, thus controlling the hash collision scale. Eventually, this trick reduces the number of parameters of the embedding layers  $U^{(2)}$  and  $U^{(3)}$ , memory usage, and increases the inference speed.

#### 4.1.2. Classification network

After obtaining the packet representation  $X_{p_i}$ , we construct the three-layer MLP network structure shown in Fig. 3 to classify it. The first layer is the input layer, which will obtain  $X_{p_i}$ . Then,  $X_{p_i}$  will be calculated through a hidden layer, viz., a fully connected network (FC), and the output of it can be expressed as Eq. (10).

$$f^{(1)}(X_{p_i}) = \text{GELU}(\text{BN}(\mathbf{W}^{(1)} \cdot X_{p_i} + \mathbf{b}^{(1)})) \quad (10)$$

**Table 3**  
Details of FastTraffic classification model.

| Function               | Layer        | Input                                       | Dropout       | Output                            |
|------------------------|--------------|---------------------------------------------|---------------|-----------------------------------|
| Packet Embedding       | Uni-gram     | $\mathcal{L}$                               | –             | $\mathcal{L} \times \mathcal{D}$  |
|                        | 2-gram       | $\mathcal{L}$                               | –             | $\mathcal{L} \times \mathcal{D}$  |
|                        | 3-gram       | $\mathcal{L}$                               | –             | $\mathcal{L} \times \mathcal{D}$  |
| Representation         | Concat       | $[\mathcal{L} \times \mathcal{D}] \times 3$ | –             | $\mathcal{L} \times 3\mathcal{D}$ |
|                        | Mean/Input   | $\mathcal{L} \times 3\mathcal{D}$           | –             | $3\mathcal{D}$                    |
| Classification Network | Hidden layer | $3\mathcal{D}$                              | –             | $\mathcal{H}$                     |
|                        | BN           | $\mathcal{H}$                               | –             | $\mathcal{H}$                     |
|                        | GELU         | $\mathcal{H}$                               | –             | $\mathcal{H}$                     |
|                        | Dropout      | $\mathcal{H}$                               | $\mathcal{R}$ | $\mathcal{H}$                     |
|                        | Output       | $\mathcal{H}$                               | –             | $ Y $                             |

$$GELU(x) = 0.5x \left( 1 + \tanh \left[ \sqrt{2/\pi} (x + cx^3) \right] \right) \quad (11)$$

$$c = 0.044715$$

Where  $\mathbf{W}^{(1)} \in \mathbb{R}^{\mathcal{H} \times 3\mathcal{D}}$  and  $\mathbf{b}^{(1)} \in \mathbb{R}^{\mathcal{H}}$  are the trainable parameter weight and bias of the hidden layer. BN is the batch normalization algorithm. The GELU activation function is used to introduce nonlinearities. Compared with the commonly used ReLU activation function, the GELU is smoother and can effectively improve the model performance [39].

Afterward, as shown in Eq. (12), we employ a Dropout strategy to alleviate the over-fitting phenomenon during the network forward propagation process, and the dropout rate is set to  $\mathcal{R}$ , where  $\mathcal{R}$  is a hyperparameter that needs to be chosen.

$$f^{(2)} = \text{Dropout}(f^{(1)}(\mathbf{X}_{p_i}), \mathcal{R}) \quad (12)$$

The last layer is the output layer of FastTraffic. This layer will calculate the final prediction of  $\mathbf{X}_{p_i}$  through the Softmax function, and  $f^{(3)} \in \mathbb{R}^{|Y|}$ ,  $|Y|$  is the size of the classification label set, as shown in Eq. (13)–(14).

$$f^{(3)} = \mathbf{W}^{(2)} \cdot f^{(2)} + \mathbf{b}^{(2)} \quad (13)$$

$$\hat{y}_{p_i} = \text{Softmax}(f^{(3)}) \quad (14)$$

Where  $\hat{y}_{p_i}$  denotes the predicted probability vector.  $\hat{y}_{p_{ic}}$  is the  $c$ th element of  $\hat{y}_{p_i}$ , which represent the probability of  $\mathbf{X}_{p_i}$  belongs to  $c$ th class in  $Y$ . During the training process, FastTraffic uses the Cross entropy loss function to measure the distance between the predicted label and the true label and then iterate the model, as defined in Eq. (15).

$$\text{loss} = -\frac{1}{|B|} \sum_{i=1}^{|B|} \sum_{j=0}^{|Y|-1} y_{p_{ic}} \log(\hat{y}_{p_{ic}}) \quad (15)$$

Where  $|B|$  is the training batch size, and  $y_{ic} = 0/1$  is the ground truth label. Overall, the details of FastTraffic classification model and the specific parameters are illustrated in Table 3.

Besides the above model structure and execution process, we use the Xavier method to initialize the model training parameters to ensure stable and fast convergence of the model, and the Adam optimizer, which is an adaptive and little memory consumption optimization algorithm, is also used in FastTraffic. The remaining hyperparameter tuning process, such as batch size, and epoch number, will be discussed detailedly in Section 5.

#### 4.2. Model complexity analysis

To demonstrate the model complexity of FastTraffic, we introduce and reproduce eight typical state-of-the-art ETC methods including six conventional methods and two lightweight methods, which are as follows:

**Table 4**  
Comparison of core structure complexity.

| Method                               | Core structure | Complexity per layer                                   | Sequential operations      | Maximum path length          |
|--------------------------------------|----------------|--------------------------------------------------------|----------------------------|------------------------------|
| 1DCNN<br>TSCRNN<br>DeepPacket<br>SAM | CNN            | $\mathcal{O}(k \cdot \mathcal{L} \cdot \mathcal{D}^2)$ | $\mathcal{O}(1)$           | $\mathcal{O}(\mathcal{L}/k)$ |
| LSTM-Att<br>TSCRNN                   | RNN            | $\mathcal{O}(\mathcal{L} \cdot \mathcal{D}^2)$         | $\mathcal{O}(\mathcal{L})$ | $\mathcal{O}(\mathcal{L})$   |
| SAM                                  | SA             | $\mathcal{O}(\mathcal{L}^2 \cdot \mathcal{D})$         | $\mathcal{O}(1)$           | $\mathcal{O}(1)$             |
| MATEC<br>ET-BERT                     | MHA            | $\mathcal{O}(h \cdot \mathcal{L}^2 \cdot \mathcal{D})$ | $\mathcal{O}(1)$           | $\mathcal{O}(1)$             |
| Datanet<br>FastTraffic               | MLP            | $\mathcal{O}(\mathcal{L} \cdot \mathcal{H})$           | $\mathcal{O}(1)$           | $\mathcal{O}(1)$             |

- 1DCNN [19] is a session-level method. The authors convert the session bytes to MNIST grayscale images, then propose 1DCNN to classify them.
- TSCRNN [11] is a flow-level method. The authors use the first 15 packets of each flow, and the length of each packet is controlled in MTU. It proposes a 2-layer 1DCNN with a bidirectional LSTM classification model.
- BiLSTM-Attention [20] (abbr. LSTM-att) is a flow-level method. The authors use the first 10 packets of each flow, and the length of each packet is controlled in MTU. It proposes a bidirectional LSTM with additive attention mechanism classification model.
- DeepPacket [9] is a packet-level method and controls packet length in MTU. It proposes a classification model based on 1DCNN.
- SAM [23] is a packet-level method and controls packet length at the first 50 bytes. It proposes a classification model based on the self-attention mechanism with a two-layer 1DCNN.
- ET-BERT [13] is a packet-level method and controls packet length at the first 512 bytes. It proposes a classification model based on 12 layers Transformer encoder and it is a method with the best classifying performance in the ETC field currently.
- MATEC [21] is a **lightweight** flow-level method. The authors use the first 3 packets of each flow, and the length of each packet is controlled in 784 bytes. The location and length of the packet are attached before these bytes. It proposes a classification model based on several modified Transformer encoders stacking.
- Datanet [22] is a **lightweight** packet-level method and controls packet length at the first 1480 bytes. It proposes a classification model based on a four-layer MLP.

For the above methods, we reproduce them all according to the original structure designs and maintenance the hyperparameter settings in their paper.

The existing ETC methods leverage different core structures to construct the final classification model, which is an upper bound on the complexity of the model and determines the theoretical and computational complexity. Hence, we summarize the core structures of the above methods and compare them with FastTraffic theoretically in three aspects, including computational complexity per layer, sequential operations, and maximum path length [40].

As designed in Section 4, the core structure of our FastTraffic is MLP, but contrast methods are designed based on CNN, RNN, self-attention (SA), and multi-head attention (MHA) respectively. Especially, MHA is the core structure of the Transformer-based method, such as MATEC and ET-BERT. CNN, RNN, SA, and MHA is leveraged to map one variable-length sequence of symbol representations  $(x_1, x_2, \dots, x_{\mathcal{L}})$  to another  $(z_1, z_2, \dots, z_{\mathcal{L}})$ , with  $x_i, z_i \in \mathbb{R}^{\mathcal{D}}$ . The computational complexity per layer is shown in Table 4, where  $\mathcal{L}$  is the input length,  $k$  is the kernel size of convolution,  $\mathcal{D}$  is the representation dimension and  $h$  is the number of attention head. Each layer of MLP is linearly mapped

**Table 5**

Two devices with different configurations.

| Hardware Name             | Configuration        | Setting               |
|---------------------------|----------------------|-----------------------|
| High-configuration device | Operation system     | Ubuntu 20.04.2 LTS    |
|                           | Traffic manipulation | Scapy & SplitCap      |
|                           | DL framework         | Pytorch-GPU 1.9       |
|                           | CPU                  | Intel Core i9-10900 K |
|                           | RAM                  | 64 GB DDR4            |
|                           | Storage              | 1TB SSD               |
|                           | GPU                  | GeForce RTX3080       |
| Low-configuration device  | VRAM                 | 10 GB                 |
|                           | Operation system     | Windows 10            |
|                           | DL framework         | Pytorch-CPU 1.9       |
|                           | CPU                  | Intel Core i5-7500    |
|                           | RAM                  | 8 GB DDR4             |
|                           | Storage              | 500 GB HDD            |

to the next layer by a weight vector, hence the complexity per layer of MLP is only  $\mathcal{O}(\mathcal{L} \cdot H)$ .  $H$  is commonly less than  $D^2$  or  $\mathcal{L} \times D$ , so the computational complexity per layer of MLP is much lower compared to CNN, RNN, SA, and MHA. Meanwhile, towards MLP, CNN, and MHA, parallel operations can be performed using vector multiplication, so their sequential operation complexity of them is only  $\mathcal{O}(1)$ , while RNN is  $\mathcal{O}(\mathcal{L})$  due to its time-step sequential execution.

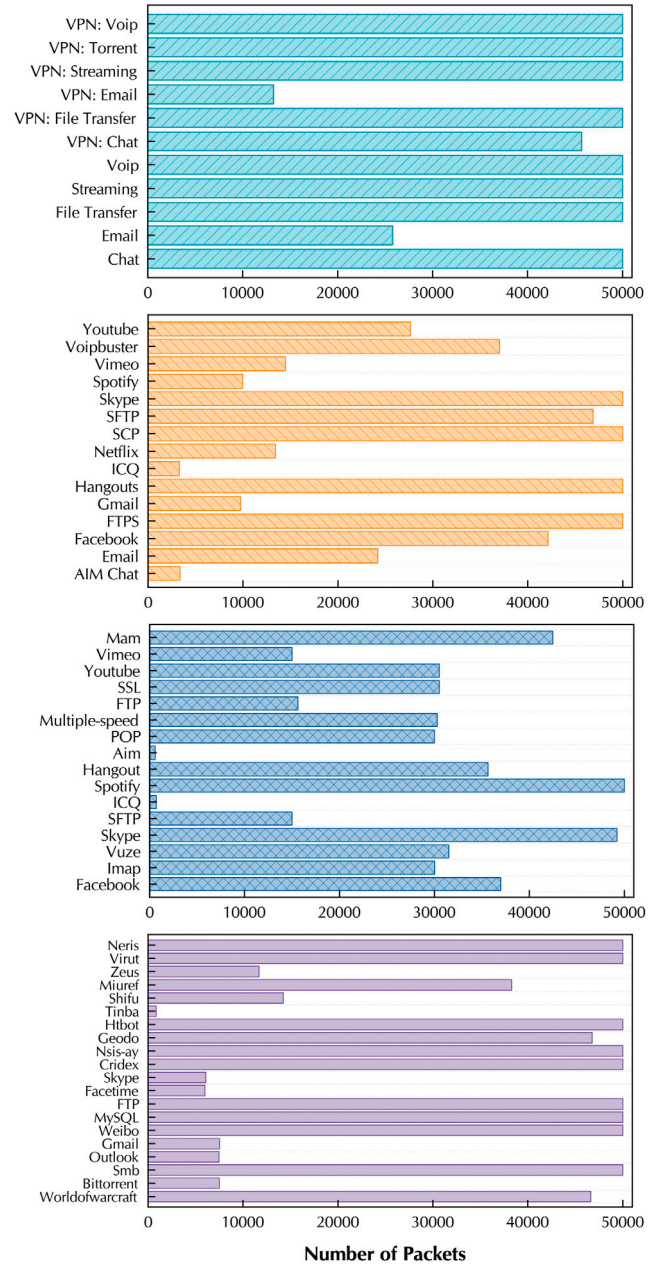
The maximum path length is defined as the length of the paths that the forward and backward information pass through in the network. The shorter these paths between any combination of neurons for input and output, the easier it is to learn long-distance dependencies, which also means less information loss between neurons [40]. So for SA, MHA, and MLP, they can connect all neurons with a constant number of sequentially executed operations, i.e.,  $\mathcal{O}(1)$ , but RNN requires  $\mathcal{O}(\mathcal{L})$  operations. CNN needs to stack  $\mathcal{O}(\mathcal{L}/k)$  convolutional layer operations with kernel size  $k$  to connect all input and output neurons, so its maximum path length is  $\mathcal{O}(\mathcal{L}/k)$ . To summarize, the computational complexity of FastTraffic is lower than other methods in three theoretical aspects. As a result, the above theoretical analysis demonstrate that FastTraffic obtains extremely low model complexity.

## 5. Experiment setup

Experiments are carefully designed to evaluate FastTraffic by comparing it with eight state-of-the-art ETC methods. In this section, we introduce the details of the experimental setup, including hardware configuration, classification tasks, evaluation metrics, and model hyperparameter tuning.

### 5.1. Device configuration & task settings

As shown in Table 5, we conduct comparative experiments on two computers with different hardware configurations. A computer equipped with an advanced GPU is used as a high-configuration device, while another ordinary computer is used as a low-configuration device. Since GPUs can accelerate the training and prediction process of deep learning models, researchers usually choose some devices with advanced GPUs for experiments in previous work. In our experiments, we follow this setting for method comparison. However, on the Internet, ISPs have not deployed network devices equipped with GPUs on a large scale due to their high prices and huge energy consumption. Therefore, ETC methods should adapt to low-configuration network devices to achieve large-scale deployment. Based on the above analysis, we have launched a survey of network devices with traffic access control and interception functions on the market, such as Sangfor AC-1000-B1200 and Cisco N540-FH-CSR-SYS. These devices generally have only one multi-core processor, 4 G or 8 G RAM, and power within 200 W. None of them have a GPU installed. In experiments, we use an ordinary computer that meets the above configuration as a low-configuration

**Fig. 5.** Number of packets for four tasks.

network device to test the performance of all ETC methods. Table 5 shows the details of this device. Since no GPU is installed, all ETC models can only run on the CPU, which is consistent with the results of our survey of mainstream network equipment. In Section 6.2, we will comprehensively compare the operating efficiencies of 9 ETC methods on this device.

According to the processing method introduced in Section 3, we constructed four sub-datasets corresponding to different classification tasks based on the three benchmark datasets. As shown in Fig. 5, both T1 and T2 classify the network traffic of ISCX-VPN. T1 is a multi-classification of network service types, while T2 is a multi-classification of different network applications. T3 is set for multi-classification of Tor's application traffic in ISCX-Tor, while T4 is a multi-classification of different types of software in USTC-TFC. Notably, the classification metrics will be evaluated based on these four sub-datasets by 10-fold cross-validation.



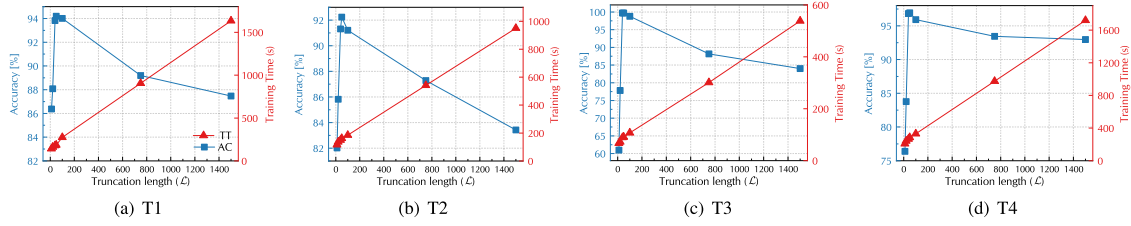


Fig. 6. Impact of truncation length  $\mathcal{L}$  on accuracy and training time of FastTraffic on four experiments.

## 5.2. Evaluation metrics

In experiments, we introduce multiple metrics to evaluate the classification performance and model efficiency of FastTraffic and other state-of-art methods.

### 5.2.1. Classification metrics

For a fair comparison, four typical classification evaluation metrics are introduced, including accuracy (AC), precision (PR), recall (RC), and F1 score (F1). As shown in Eq. (16), AC is equal to the number of samples accurately classified divided by the total number of packets. Besides, in order to evaluate the classification results of all categories as a whole, we use the macro-average method to calculate the values of PR, RC, and F1. Their calculations are described in Eqs. (17) to (19).

$$AC = \frac{TP + TN}{TP + TN + FP + FN} \quad (16)$$

$$PR = \frac{1}{|Y|} \sum_{i=0}^{|Y|-1} \frac{TP_i}{TP_i + FP_i} \quad (17)$$

$$RC = \frac{1}{|Y|} \sum_{i=0}^{|Y|-1} \frac{TP_i}{TP_i + FN_i} \quad (18)$$

$$F1 = \frac{2 \times PR \times RC}{PR + RC} \quad (19)$$

### 5.2.2. Efficiency metrics

In addition to classification performance, operational efficiency is another important dimension to evaluate ETC methods. Network traffic classification usually consists of three steps: traffic capture, data preprocessing, and model classification. According to [41,42], the process of traffic capture can be accelerated by FPGA-based devices, so the bottleneck of the ETC method mainly lies in the last two steps.

To objectively compare the efficiency of these ETC methods, we introduce two metrics,  $PT$  and  $IT$ .  $PT$  describes the preprocessing time consumed by one packet, and  $IT$  represents the inference time of the classification model for one sample. It is worth noting that both  $PT$  and  $IT$  are the average results of 10,000 random samples under 20 repeated experiments. To obtain accurate results of  $PT$  and  $IT$ , we load the models into the memory at one time. The models are resident in the memory during each repeated experiments, thus avoiding the additional time overhead introduced by loading from disk multiple times. Furthermore, we define throughput ( $TH$ ) based on  $PT$  and  $IT$ . According to Eq. (20), it represents the number of packets processed by the classification model per second.

$$TH = \frac{1}{PT + IT} \quad (20)$$

## 5.3. Hyperparameter tuning

To achieve the best performance of the FastTraffic model, we perform hyperparameter tuning on *ISCX-VPN*. In experiments, we first take AC as the evaluation metric and T1 as the target task and then use the random search method [43] to determine the optimal values of these hyperparameters within the search ranges listed in Table 6.

Table 6

Hyperparameter tuning for FastTraffic.

| Hyperparameters     | Search range      | Final selection |
|---------------------|-------------------|-----------------|
| Truncation length   | [50, ..., 100]    | 50              |
| Embedding dimension | [40, 50, 60, 70]  | 40              |
| Hidden layer size   | [64, ..., 200]    | 150             |
| Dropout rate        | [0, ..., 0.5]     | 0.38            |
| Training epoch      | [5, ..., 30]      | 24              |
| Batch size          | [64, ..., 256]    | 204             |
| Learning rate       | [1E-4, ..., 1E-2] | 3.6E-3          |

The truncation length  $\mathcal{L}$  of the original data packet is an important parameter that affects the classification performance and operational efficiency of FastTraffic. In Section 3.2, we propose an interval-based searching strategy and decide to tune  $\mathcal{L}$  within the interval [50, 100]. According to the results of the above experiments, 50 bytes is the optimal value of  $\mathcal{L}$ . In order to further verify the rationality of this value, we conduct experiments on four classification tasks. We fix the other hyperparameters and let  $\mathcal{L}$  take the values in the sequence {10, 20, 40, 50, 100, 750, 1500} to explore the influence of  $\mathcal{L}$  on classification accuracy and training time. As shown in Fig. 6, FastTraffic achieves the highest accuracy in all tasks when  $\mathcal{L}$  is 50 bytes. In terms of operational efficiency, as the value of  $\mathcal{L}$  increases, the training time of the model increases linearly. Besides,  $IT$  is only 0.43 ms when  $\mathcal{L}$  is 50 bytes, but it grows to 1.41 ms when  $\mathcal{L}$  is 1500 bytes. Therefore, setting a larger  $\mathcal{L}$  does not bring better classification performance but introduces more time overhead. To sum up, we fix  $\mathcal{L}$  to be 50 bytes.

## 6. Experimental results & analysis

In experiments, we compare the classification performance and operational efficiency of FastTraffic and eight state-of-the-art ETC methods, evaluate their performance in low-resource scenarios, and finally prove the rationality of the FastTraffic model design through an ablation study.

### 6.1. Classification performance comparison

Tables 7 and 8 depict the classification performance of all ETC methods. To illustrate the feasibility of FastTraffic, we first compare it with six conventional methods. In T1 and T2, it far outperforms the five ETC methods and is only worse than ET-BERT. According to the results in Table 7, the F1 score of FastTraffic exceed 93% in the above two tasks. As shown in Table 8, almost all methods achieve satisfactory performance in T3, and the F1 score of FastTraffic reaches 99.40%. It is worth noting that since 1DCNN is a session-level method and the dataset *ISCX-Tor* cannot provide a sufficient number of samples to train the model, it performs very poorly. For T4, FastTraffic's classification performance is relatively weaker than other methods, but its F1 is still acceptable, exceeding 95%. ET-BERT is the best of all methods due to its pre-training strategy and complex model, while overall FastTraffic achieves the second-best performance after ET-BERT. It is worth noting that the above six conventional methods are designed to pursue high classification performance and lack lightweight considerations.

**Table 7**

Classification performance comparison among nine ETC methods on T1, and T2. Results are in the format Avg.(±Std.) obtained over 10-fold.

| Task               | T1 [%]              |                     |                     |                     | T2 [%]              |                     |                     |                     |
|--------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|
| Method             | AC                  | PR                  | RC                  | F1                  | AC                  | PR                  | RC                  | F1                  |
| 1DCNN              | 91.52(±0.13)        | 93.12(±0.15)        | 93.11(±0.22)        | 93.09(±0.18)        | 91.78(±1.12)        | 86.08(±1.63)        | 79.47(±1.57)        | 81.83(±1.82)        |
| TSCRNN             | 88.04(±1.02)        | 89.36(±0.97)        | 89.41(±1.54)        | 89.26(±1.22)        | 80.25(±1.00)        | 77.59(±1.73)        | 76.11(±1.56)        | 76.21(±1.48)        |
| LSTM-Att           | 88.01(±1.03)        | 89.38(±0.97)        | 89.03(±1.10)        | 88.92(±1.11)        | 80.54(±1.02)        | 79.16(±2.20)        | 74.55(±1.10)        | 76.05(±0.70)        |
| DeepPacket         | 90.66(±1.19)        | 91.18(±1.37)        | 89.50(±1.31)        | 89.83(±1.33)        | 89.05(±0.32)        | 88.67(±0.53)        | 86.68(±0.89)        | 87.23(±0.73)        |
| SAM                | 91.59(±0.17)        | 92.46(±0.35)        | 91.99(±0.18)        | 92.00(±0.17)        | 91.02(±0.22)        | 93.56(±0.60)        | 92.27(±0.70)        | 92.72(±0.58)        |
| ET-BERT            | 99.92(±0.09)        | 99.98(±0.04)        | 99.95(±0.06)        | 99.95(±0.05)        | 98.97(±0.13)        | 98.96(±0.36)        | 99.11(±0.12)        | 99.02(±0.23)        |
| MATEC              | 73.20(±0.72)        | 84.43(±1.04)        | 82.40(±1.03)        | 82.87(±0.81)        | 69.21(±0.69)        | 73.32(±1.47)        | 65.40(±2.03)        | 68.24(±1.43)        |
| Datanet            | 91.85(±0.31)        | 91.57(±0.40)        | 91.06(±0.30)        | 91.17(±0.32)        | 87.19(±0.44)        | 84.43(±1.09)        | 82.85(±0.95)        | 83.16(±0.75)        |
| <b>FastTraffic</b> | <b>94.50(±0.13)</b> | <b>94.77(±0.23)</b> | <b>94.26(±0.08)</b> | <b>94.40(±0.12)</b> | <b>92.24(±0.54)</b> | <b>93.58(±0.33)</b> | <b>92.84(±0.49)</b> | <b>93.12(±0.35)</b> |

**Table 8**

Classification performance comparison among nine ETC methods on T3, and T4. Results are in the format Avg.(±Std.) obtained over 10-fold.

| Task               | T3 [%]              |                     |                     |                     | T4 [%]       |              |              |              |
|--------------------|---------------------|---------------------|---------------------|---------------------|--------------|--------------|--------------|--------------|
| Method             | AC                  | PR                  | RC                  | F1                  | AC           | PR           | RC           | F1           |
| 1DCNN              | 58.78(±0.26)        | 62.83(±3.06)        | 43.81(±0.41)        | 46.09(±0.83)        | 98.68(±0.20) | 98.96(±0.27) | 98.96(±0.25) | 98.95(±0.27) |
| TSCRNN             | 98.97(±0.31)        | 94.83(±2.61)        | 94.73(±2.45)        | 94.01(±2.40)        | 96.90(±0.43) | 96.95(±0.40) | 96.86(±0.42) | 96.85(±0.43) |
| LSTM-Att           | 97.30(±0.71)        | 95.87(±1.02)        | 95.36(±1.42)        | 95.30(±1.37)        | 97.01(±0.22) | 97.03(±0.19) | 96.89(±0.23) | 96.92(±0.22) |
| DeepPacket         | 98.78(±0.11)        | 95.08(±0.88)        | 94.01(±0.71)        | 94.29(±0.71)        | 95.36(±0.29) | 95.89(±0.80) | 93.23(±1.20) | 93.88(±1.20) |
| SAM                | 98.05(±1.35)        | 96.74(±1.67)        | 96.44(±1.80)        | 96.27(±1.94)        | 95.73(±1.53) | 96.70(±1.15) | 96.61(±1.21) | 96.64(±1.19) |
| ET-BERT            | 99.92(±0.09)        | 99.98(±0.04)        | 99.95(±0.06)        | 99.95(±0.05)        | 98.97(±0.13) | 98.96(±0.36) | 99.11(±0.12) | 99.02(±0.23) |
| MATEC              | 99.39(±0.10)        | 95.72(±0.49)        | 95.04(±0.57)        | 95.17(±0.55)        | 97.19(±1.11) | 97.40(±1.01) | 97.28(±1.13) | 97.26(±1.15) |
| Datanet            | 98.41(±0.25)        | 94.32(±0.63)        | 93.74(±0.87)        | 93.73(±0.99)        | 96.74(±0.19) | 97.07(±0.20) | 97.11(±0.22) | 97.05(±0.21) |
| <b>FastTraffic</b> | <b>99.72(±0.03)</b> | <b>99.47(±0.24)</b> | <b>99.33(±0.18)</b> | <b>99.40(±0.19)</b> | 96.93(±0.16) | 96.57(±0.74) | 94.99(±0.70) | 95.53(±0.42) |

In the following experiments, we will demonstrate their shortcomings in operational efficiency. Second, we compare FastTraffic with two lightweight methods, MATEC and Datanet. In all four classification tasks, FastTraffic outperforms the above two methods. Especially in **T1** and **T2**, the gap between them is significant. The above results demonstrate that FastTraffic can better complete traffic classification while pursuing lightweight.

In **Table 7**, all ETC methods perform worse on **T1** and **T2** than on **T3** and **T4**. The reason is that *ISCX-VPN* contains much more samples than other datasets, and each category contains traffic from various sources. For example, the *VoIP* category in **T1** includes the traffic flows of *Skype\_video*, *VoipBuster*, *Facebook\_audio*, *Facebook\_video*, etc. Due to differences in traffic samples of various software, some samples are mislabeled during the classification process. The complex composition of *ISCX-VPN* poses a difficult challenge, causing a drop in classification performance for all methods.

This difficulty is also reflected in the confusion matrices, as shown in **Figs. 7** and **8**. The results indicate an overall enhancement of the confusion matrices achieved by FastTraffic when compared to MATEC and Datanet. Towards **T1**, misclassification patterns are relatively similar for the three methods, with the *File Transfer* class and *VoIP* class being the most likely to be misclassified, due to their complex composition. The difference in misclassification is most pronounced on **T2**, where FastTraffic improved the MATEC's recall of the *Email* class (39.80%) to 83.64%, Datanet's recall of the *Aim Chat* class (26.89%) to 90.62%. Hence, benefiting from packet truncation and N-gram feature embedding, FastTraffic can accurately classify more packets than the other two methods. Therefore, it obtains better results than MATEC and Datanet.

## 6.2. Operational efficiency comparison

To evaluate the feasibility of deployment, we compare the operational efficiency of FastTraffic with other methods on high-configuration and low-configuration devices.

**Table 9**

Comparison of the number of parameters among nine ETC models..

| Method             | Number of parameters |
|--------------------|----------------------|
| 1DCNN              | 1.7M                 |
| TSCRNN             | 2.9M                 |
| LSTM-Att           | 1.6M                 |
| DeepPacket         | 10M                  |
| SAM                | 0.8M                 |
| ET-BERT            | 132M                 |
| MATEC              | 2.6M                 |
| Datanet            | 0.19M                |
| <b>FastTraffic</b> | <b>0.43M</b>         |

### 6.2.1. Number of model parameters

The number of parameters is an important metric that affects the operational efficiency of a deep learning model. We use Pytorch API to count the parameters of all ETC models. As shown in **Table 9**, FastTraffic's number of parameters is 0.43M and is the second least among all methods. Compared with the six conventional methods, FastTraffic requires half or even fewer parameters of them. Although proposed as a lightweight method, MATEC has six times more parameters than FastTraffic. Due to the N-gram feature embedding, FastTraffic has slightly more parameters than Datanet, but it obtains better performance in **Tables 7** and **8**. Therefore, FastTraffic can achieve the goal of the lightweight model while ensuring its classification performance. In addition, although ET-BERT achieves the best classification performance, the pre-training process and complex model make it harvest up to 132M parameters. Therefore, it is impossible to deploy ET-BERT on resource-constrained devices in real network scenarios.

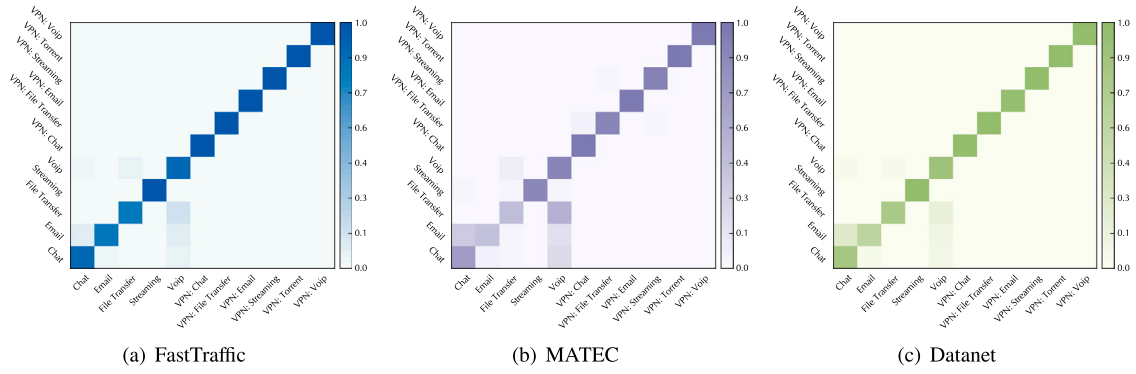


Fig. 7. Confusion matrices of three lightweight methods on T1.

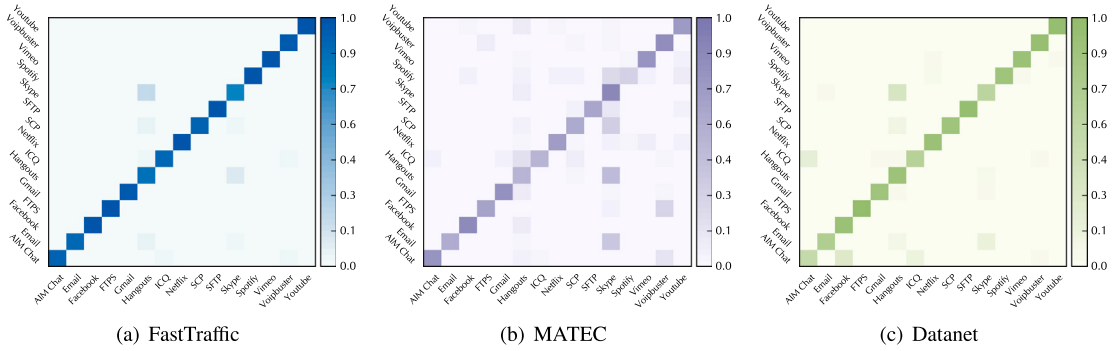


Fig. 8. Confusion matrices of three lightweight methods on T2.

**Table 10**  
Operational efficiency comparison among nine ETC methods on the high-configuration device.

| Method      | PT(ms) | IT(ms) | TH       |
|-------------|--------|--------|----------|
| 1DCNN       | 0.87   | 0.64   | 662 sps  |
| TSCRNN      | 1.3    | 5.3    | 152 fps  |
| LSTM-Att    | 1.7    | 2.6    | 233 fps  |
| DeepPacket  | 0.61   | 1.0    | 62 pps   |
| SAM         | 0.49   | 0.96   | 690 pps  |
| ET-BERT     | 0.61   | 9.99   | 94 pps   |
| MATEC       | 1.3    | 1.3    | 385 fps  |
| Datanet     | 0.27   | 0.51   | 1282 pps |
| FastTraffic | 0.59   | 0.39   | 1010 pps |

### 6.2.2. Efficiency on high-configuration device

We evaluate *PT*, *IT*, and *TH* of all ETC methods on the high-configuration device. As shown in Table 10, FastTraffic obtains a *PT* of 0.59ms, an *IT* of 0.39ms, and achieves a *TH* of 1010 pps. It is the second-best result among all methods. The complex model brings ET-BERT the best classification performance but causes the lowest operating efficiency. Although they are both packet-level ETC methods, ET-BERT's *TH* is only 9.3% of FastTraffic's *TH*. As mentioned above, since MATEC's model has more parameters, its operational efficiency is worse than other lightweight methods. Datanet only uses a four-layer MLP, so it obtains the highest efficiency. FastTraffic operates slightly less efficiently than Datanet. The main reason is that FastTraffic needs to filter unrelated packets before feeding samples into the classification model. According to the discussion in Section 6.1, although FastTraffic sacrifices part of the operating efficiency, it dramatically improves the classification performance. Therefore, FastTraffic is the first choice for fast traffic classification.

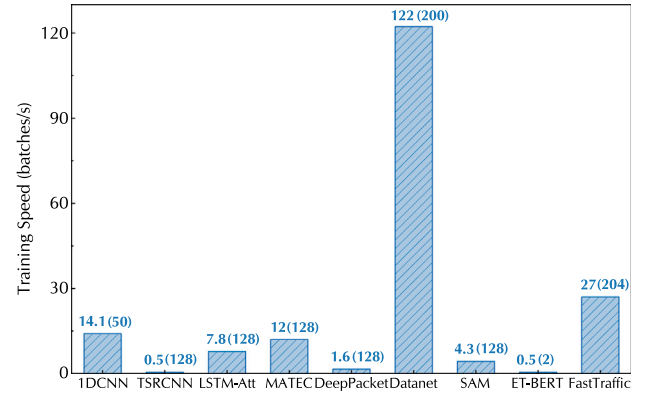


Fig. 9. Training speed comparison among nine ETC methods on the low-configuration device.

### 6.2.3. Efficiency on low-configuration device

To verify the feasibility of deploying the ETC methods in an actual network environment, we use a low-configuration computer described in Table 5 to evaluate their operating efficiency. In the following experiments, we mainly examine the operational efficiency of model training and traffic classification.

First, we introduce the training speed (*TS*) as an evaluation metric, and it is equal to the number of batches that the ETC model can process per second. Due to the frequent updates of network applications and protocols, the features of the network traffic flows change over time. Therefore, any ETC model cannot guarantee excellent classification results in long-term use after only one training. Besides, the training process of the classification models requires significant computational and storage resources. Although it is a good idea to train the models on the cloud and then distribute the parameters to devices, many ISPs still

**Table 11**  
Operational efficiency comparison among nine ETC methods on the low-configuration device.

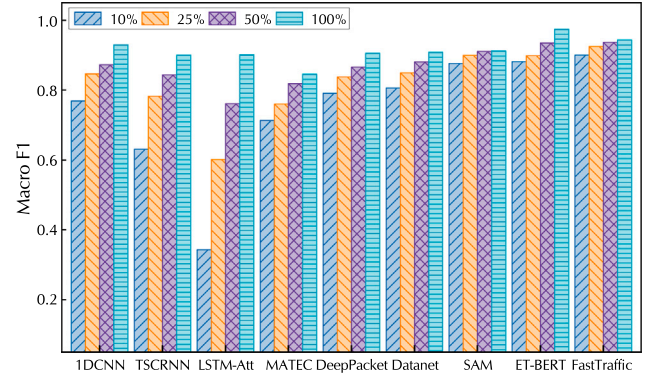
| Method      | File size (MB) | On-device training | PT(ms) | IT(ms) | TH      | CPU utilization | Memory usage(MB) |
|-------------|----------------|--------------------|--------|--------|---------|-----------------|------------------|
| 1DCNN       | 6.6            | ✓                  | 5.5    | 1.7    | 139 sps | 15% ~ 21%       | ~ 112            |
| TSCRNN      | 11.0           | ✗                  | 4.1    | 18.7   | 44 fps  | 16% ~ 23%       | ~ 119            |
| LSTM-Att    | 5.9            | ✓                  | 4.4    | 5.8    | 98 fps  | 8% ~ 16%        | ~ 78             |
| DeepPacket  | 38.5           | ✗                  | 1.3    | 4.4    | 175 pps | 16% ~ 20%       | ~ 146            |
| SAM         | 3.3            | ✗                  | 0.7    | 1.1    | 556 pps | 14% ~ 22%       | ~ 111            |
| ET-BERT     | 504            | ✗                  | 1.2    | 170.9  | 6 pps   | 33% ~ 38%       | ~ 660            |
| MATEC       | 9.9            | ✓                  | 2.4    | 2.1    | 227 fps | 8% ~ 13%        | ~ 84             |
| Datanet     | 0.74           | ✓                  | 0.53   | 0.57   | 909 pps | 8% ~ 10%        | ~ 72             |
| FastTraffic | 1.60           | ✓                  | 1.22   | 0.25   | 680 pps | 7%~11%          | ~74              |

want to do model training on their local devices due to data privacy concerns. Therefore, we use training speed to evaluate whether an ETC method can complete the training process on a low-configuration network device. In Fig. 9, FastTraffic achieves the second-fastest training speed. Compared with six conventional ETC methods, FastTraffic is more than twice as fast as them. Besides, FastTraffic is faster than MATEC but slower than Datanet. The reason is that Datanet directly sends data packets to MLP for classification, while FastTraffic needs to complete packet truncation and N-gram feature embedding before this. It also explains why FastTraffic maintains a higher classification accuracy than Datanet.

Second, we count the size of the model file. As shown in Table 11, the file size of FastTraffic is only 1.6 MB, so it will not cause storage pressure on any network device. Due to the complexity of the model, ET-BERT needs to occupy 504 MB of storage space. It is a prohibitive overhead for resource-constrained network devices. Based on the above analysis, FastTraffic's excellent operating efficiency ensures it can achieve fast model training on most network devices.

To evaluate the operational efficiency of traffic identification, we calculate the *TH*, *PT*, and *IT* of all ETC methods. As shown in Table 11, the *TH* of FastTraffic on the low-configuration device reaches 680 pps. It is lower than the result on the high-configuration device but is still the second best among all ETC methods. Since there is no GPU to accelerate the classification process, the *TH*s of these ETC methods based on deep learning models drop significantly. ET-BERT only achieves 6 pps. Although it obtains the best classification performance, its model is difficult to deploy on network devices. In addition, Table 11 shows the *PT* and *IT* of all methods. FastTraffic takes only 1.47ms to process one packet, slightly longer than Datanet's 1.1ms. It is consistent with the experimental results on the high-configuration device. FastTraffic performs packet filtering before sending samples to the classification network. Although this step will increase the value of *PT*, it can improve classification performance. Given the low execution efficiency of Python's Scapy library, there is still room to shorten the *PT* during engineering. In addition, it is worth noting that FastTraffic's *IT* is 0.25ms, which is even lower than the results on the high-configuration device. It proves that FastTraffic does not rely on high-performance GPUs during traffic identification. Therefore, FastTraffic is suitable for deployment on most mainstream network devices.

In addition to identifying traffic, network devices must provide daily services, such as packet forwarding. ETC methods cannot occupy too many device resources after deployment. We observe the memory usage and CPU utilization on the low-configuration device to compare these ETC methods. As shown in Table 11, FastTraffic only needs ~74MB of memory when performing traffic classification and takes up ~10% of the computing power of the CPU. So it meets the lightweight requirement when deployed on the current network devices. In contrast, ET-BERT uses ~660MB of memory and takes up one-third of the computing power, which will seriously affect the QoS provided by the network device.



**Fig. 10.** Comparison of low-resource learning capabilities among nine ETC methods on T1.

### 6.3. Comparison in low-resource scenarios

In real network scenarios, constructing a large-scale dataset for network applications or services requires a lot of resources and labor costs. If an ETC method only uses a small number of labeled samples to complete model training, it can reduce the actual operating cost. In [13], the authors refer to it as the low-resource learning ability of ETC methods. Accordingly, we randomly select 10%, 25%, and 50% samples from the original dataset for experiments to evaluate the low-resource learning ability of all methods. As shown in Fig. 10, the *F1* values of FastTraffic under the above sampling ratios are 90.05%, 92.48%, and 93.70%. Although the values are all lower than the result of training the model with all samples, they are still above 90%. Compared with FastTraffic, the *F1* values of other ETC methods drop more sharply when the training samples are insufficient. Besides, we note that flow-level methods are more susceptible to the problem that there are not enough training samples. Since a flow contains many packets, flow-level methods will get fewer samples for the same bytes of traffic data. Due to adopting a smaller identification granularity, FastTraffic can obtain more labeled samples, thereby training a more robust model. To sum up, it has excellent learning ability in low-resource scenarios.

### 6.4. Ablation study

To verify the validity and rationality of each component of the FastTraffic model, we conduct an ablation study on T1 and set the variants of FastTraffic for comparison as follows.

- w/  $U^{(1)}$ : We only retain the single uni-gram embedding modify the classification network detail to prove the validity of 2-gram and 3-gram design.



**Table 12**

Ablation study of FastTraffic on T1 in the format Avg.(±Std.) obtained over 10-fold.

| Method               | U <sup>(1)</sup> | U <sup>(2)</sup> | U <sup>(3)</sup> | W <sup>(1)</sup> | AC [%]       | F1 [%]       |
|----------------------|------------------|------------------|------------------|------------------|--------------|--------------|
| w/ U <sup>(1)</sup>  | ✓                | ✗                | ✗                | ✓                | 89.65(±0.18) | 89.26(±0.16) |
| w/o U <sup>(1)</sup> | ✗                | ✓                | ✓                | ✓                | 93.75(±0.24) | 93.65(±0.24) |
| w/o U <sup>(2)</sup> | ✓                | ✗                | ✓                | ✓                | 94.05(±0.23) | 93.96(±0.27) |
| w/o U <sup>(3)</sup> | ✓                | ✓                | ✗                | ✓                | 92.49(±0.13) | 92.35(±0.13) |
| w/o W <sup>(1)</sup> | ✓                | ✓                | ✓                | ✗                | 93.49(±0.09) | 93.37(±0.08) |
| Fast Traffic         | ✓                | ✓                | ✓                | ✓                | 94.50(±0.13) | 94.40(±0.12) |

- w/o U<sup>(1)</sup> ~ w/o U<sup>(3)</sup>: We discard the single uni-gram, 2-gram, and 3-gram embedding layers in order and correspondingly modify the classification network.
- w/o W<sup>(1)</sup>: We remove the hidden layer and GELU activation function and only retain the output layer and Softmax function for classification.

From the results in Table 12, we get the following three observations:

- (1) w/ U<sup>(1)</sup> achieves the worst performance since its model uses only uni-gram embedding, omitting the structural and sequential information provided by 2-gram and 3-gram feature embedding. This result proves that introducing 2-gram and 3-gram embedding can help improve classification performance.
- (2) w/o U<sup>(1)</sup> ~ w/o U<sup>(3)</sup> illustrates that each embedding discard has a negative impact on the model performance and fully illustrates the effectiveness of the n-gram feature embedding design. It is worth noting that w/o U<sup>(3)</sup> causes a 2% decrease in AC and F1, which indicates that 3-gram embedding is the most important module. It can extract more useful information from packets than the uni-gram and 2-gram embedding.
- (3) w/o W<sup>(1)</sup> proves that the design of the hidden layer is valid and reasonable. We further extract latent information from packet representation through the hidden layer and use a non-linear activation function to improve classification performance.

In summary, each component of the model contributes to the improvement of classification performance. The non-redundant model structure ensures the achievement of FastTraffic's lightweight goal.

## 7. Conclusion

In this paper, we propose FastTraffic, a lightweight method for fast and accurate encrypted traffic classification on network devices. In the data preprocessing, we treat each IP packet as a text-like byte sequence and truncate the first 50 bytes of the IP packet for efficient data representation. In the model design, we propose an N-gram embedding method to extract the structural and sequential information in raw bytes of packets and use a three-layer MLP to perform fast classification. Through theoretical analysis and experimental verification, we prove that FastTraffic achieves the lightweight goal and obtains the second-best classification performance among nine ETC methods. Experimental results show that FastTraffic reaches a throughput of 680 pps on low-configuration network devices with low CPU and memory usage. In a word, FastTraffic can achieve encrypted traffic classification on mainstream network devices and obtain excellent classification performance and high operational efficiency. In the future, we will still aim at the lightweight ETC method and plan to deepen our research in two directions: traffic data representation and classification model construction.

## CRedit authorship contribution statement

**Yuwei Xu:** Conceptualization, Writing – review & editing, Funding acquisition. **Jie Cao:** Methodology, Software, Validation, Writing – original draft. **Kehui Song:** Resources, Visualization, Investigation. **Qiao Xiang:** Formal analysis, Supervision. **Guang Cheng:** Project administration, Supervision.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

The three datasets used in this paper are all publicly available and all cite corresponding references.

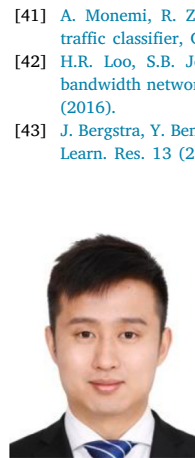
## Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant U22B2025, and in part by the National Key R&D Program of China under Grant 2020YFB1005500.

## References

- [1] L. Encrypt, Percentage of web pages loaded by firefox using HTTPS, 2023, URL <https://letsencrypt.org/stats/>.
- [2] E. Papadogiannaki, S. Ioannidis, A survey on encrypted network traffic analysis applications, techniques, and countermeasures, *ACM Comput. Surv.* 54 (6) (2021) 1–35.
- [3] S. Rezaei, X. Liu, Deep learning for encrypted traffic classification: An overview, *IEEE Commun. Mag.* 57 (5) (2019) 76–81.
- [4] S.-J. Xu, G.-G. Geng, X.-B. Jin, D.-J. Liu, J. Weng, Seeing traffic paths: Encrypted traffic classification with path signature features, *IEEE Trans. Inf. Forensics Secur.* (2022).
- [5] T. Shapira, Y. Shavitt, FlowPic: A generic representation for encrypted traffic classification and applications identification, *IEEE Trans. Netw. Serv. Manag.* 18 (2) (2021) 1218–1232.
- [6] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, K. Wehrle, Website fingerprinting at internet scale, in: NDSS, 2016.
- [7] M. Shen, Y. Liu, L. Zhu, K. Xu, X. Du, N. Guizani, Optimizing feature selection for efficient encrypted traffic classification: A systematic approach, *IEEE Netw.* 34 (4) (2020) 20–27.
- [8] T. Zhang, H. Qiu, M. Mellia, Y. Li, H. Li, K. Xu, Interpreting AI for networking: Where we are and where we are going, *IEEE Commun. Mag.* 60 (2) (2022) 25–31.
- [9] M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, M. Saberian, Deep packet: A novel approach for encrypted traffic classification using deep learning, *Soft Comput.* 24 (3) (2020) 1999–2012.
- [10] X. Ren, H. Gu, W. Wei, Tree-RNN: Tree structural recurrent neural network for network traffic classification, *Expert Syst. Appl.* 167 (2021) 114363.
- [11] K. Lin, X. Xu, H. Gao, TSCRNN: A novel classification scheme of encrypted traffic based on flow spatiotemporal features for efficient management of IIoT, *Comput. Netw.* 190 (2021) 107974.
- [12] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, J. Lloret, Network traffic classifier with convolutional and recurrent neural networks for Internet of Things, *IEEE Access* 5 (2017) 18042–18050.
- [13] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, J. Yu, ET-BERT: A contextualized datagram representation with pre-training transformers for encrypted traffic classification, in: *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642.
- [14] Z. Meng, M. Wang, J. Bai, M. Xu, H. Mao, H. Hu, Interpreting deep learning-based networking systems, in: *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, 2020, pp. 154–171.
- [15] M. Shen, Y. Liu, L. Zhu, X. Du, J. Hu, Fine-grained webpage fingerprinting using only packet length information of encrypted traffic, *IEEE Trans. Inf. Forensics Secur.* 16 (2020) 2046–2059.
- [16] R. Yuan, Z. Li, X. Guan, L. Xu, An SVM-based machine learning method for accurate internet traffic classification, *Inform. Syst. Front.* 12 (2) (2010) 149–156.
- [17] S. Feghhi, D.J. Leith, A web traffic analysis attack using only timing information, *IEEE Trans. Inf. Forensics Secur.* 11 (8) (2016) 1747–1759.

- [18] R. Dubin, A. Dvir, O. Pele, O. Hadar, I know what you saw last minute—encrypted http adaptive video streaming title classification, *IEEE Trans. Inf. Forensics Secur.* 12 (12) (2017) 3039–3049.
- [19] W. Wang, M. Zhu, J. Wang, X. Zeng, Z. Yang, End-to-end encrypted traffic classification with one-dimensional convolution neural networks, in: 2017 IEEE International Conference on Intelligence and Security Informatics, ISI, IEEE, 2017, pp. 43–48.
- [20] H. Yao, C. Liu, P. Zhang, S. Wu, C. Jiang, S. Yu, Identification of encrypted traffic through attention mechanism based long short term memory, *IEEE Trans. Big Data* (2019).
- [21] J. Cheng, Y. Wu, E. Yuepeng, J. You, T. Li, H. Li, J. Ge, MATEC: A lightweight neural network for online encrypted traffic classification, *Comput. Netw.* 199 (2021) 108472.
- [22] P. Wang, F. Ye, X. Chen, Y. Qian, Datanet: Deep learning based encrypted network traffic classification in sdn home gateway, *IEEE Access* 6 (2018) 55380–55391.
- [23] G. Xie, Q. Li, Y. Jiang, Self-attentive deep learning method for online traffic classification and its interpretability, *Comput. Netw.* 196 (2021) 108267.
- [24] B. Saltaformaggio, H. Choi, K. Johnson, Y. Kwon, Q. Zhang, X. Zhang, D. Xu, J. Qian, Eavesdropping on {fine-grained} user activities within smartphone apps over encrypted network traffic, in: 10th USENIX Workshop on Offensive Technologies, WOOT 16, 2016.
- [25] J. Devlin, M.-W. Chang, K. Lee, K. Toutanova, Bert: Pre-training of deep bidirectional transformers for language understanding, 2018, arXiv preprint arXiv:1810.04805.
- [26] S. Rezaei, X. Liu, Multitask learning for network traffic classification, in: 2020 29th International Conference on Computer Communications and Networks, ICCCN, IEEE, 2020, pp. 1–9.
- [27] H. Huang, H. Deng, J. Chen, L. Han, W. Wang, Automatic multi-task learning system for abnormal network traffic detection, *Int. J. Emerg. Technol. Learn.* 13 (4) (2018).
- [28] X. Wang, S. Chen, J. Su, App-net: A hybrid neural network for encrypted mobile traffic classification, in: IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops, INFOCOM WKSHPS, IEEE, 2020, pp. 424–429.
- [29] G. Aceto, D. Ciunzio, A. Montieri, A. Pescapé, MIMETIC: Mobile encrypted traffic classification using multimodal deep learning, *Comput. Netw.* 165 (2019) 106944.
- [30] G. Aceto, D. Ciunzio, A. Montieri, A. Pescapé, DISTILLER: Encrypted traffic classification via multimodal multitask deep learning, *J. Netw. Comput. Appl.* 183 (2021) 102985.
- [31] A. Nascita, A. Montieri, G. Aceto, D. Ciunzio, V. Persico, A. Pescapé, Improving performance, reliability, and feasibility in multimodal multitask traffic classification with XAI, *IEEE Trans. Netw. Serv. Manag.* (2023).
- [32] G. Draper-Gil, A.H. Lashkari, M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and vpn traffic using time-related, in: Proceedings of the 2nd International Conference on Information Systems Security and Privacy, ICISPP, 2016, pp. 407–414.
- [33] A.H. Lashkari, G. Draper-Gil, M.S.I. Mamun, A.A. Ghorbani, et al., Characterization of tor traffic using time based features, in: Proceedings of the 3rd International Conference on Information Systems Security and Privacy, ICISPP, 2017, pp. 253–262.
- [34] W. Wang, M. Zhu, X. Zeng, X. Ye, Y. Sheng, Malware traffic classification using convolutional neural network for representation learning, in: 2017 International Conference on Information Networking, ICOIN, IEEE, 2017, pp. 712–717.
- [35] G. Aceto, D. Ciunzio, A. Montieri, A. Pescapé, Mobile encrypted traffic classification using deep learning: Experimental evaluation, lessons learned, and challenges, *IEEE Trans. Netw. Serv. Manag.* 16 (2) (2019) 445–458.
- [36] O. Barut, Y. Luo, P. Li, T. Zhang, R1DIT: Privacy-preserving malware traffic classification with attention-based neural networks, *IEEE Trans. Netw. Serv. Manag.* (2022).
- [37] T. Mikolov, K. Chen, G. Corrado, J. Dean, Efficient estimation of word representations in vector space, 2013, arXiv preprint arXiv:1301.3781.
- [38] P. Bojanowski, E. Grave, A. Joulin, T. Mikolov, Enriching word vectors with subword information, *Trans. Assoc. Comput. Linguist.* 5 (2017) 135–146.
- [39] D. Hendrycks, K. Gimpel, Gaussian error linear units (GELUS), 2016, arXiv preprint arXiv:1606.08415.
- [40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, Attention is all you need, in: *Advances in Neural Information Processing Systems*, Vol. 30, 2017.



**Yuwei Xu** received the B.Eng. degree in information security and the Ph.D. degree in computer science from Nankai University, Tianjin, China, in 2007 and 2012, respectively. He worked as an Assistant Professor with the College of Computer and Control Engineering, from 2012 to 2017, and an Associate Professor with the College of Cyber Science, Nankai University, from 2018 to 2019. He is currently an Associate Professor with the School of Cyber Science and Engineering, Southeast University, Nanjing, China. His research interests include the Internet of Things, future network architecture, and network security.



**Jie Cao** received B.S. degree in Information Security from Xi'an University of Posts and Telecommunications, Xi'an, China. He is currently pursuing M.S. degree in Cyberspace Security from Southeast University, Nanjing, China. His research interests mainly revolve around the field of cybersecurity and encrypted traffic identification.



**Kehui Song** received the B.S., M.S., and Ph.D. degrees in computer science from Nankai University. She is currently working as a Post-Doctoral Researcher with the College of Computer Science, Nankai University. Her main research interests include machine learning, medical data analysis, and big data management.



**Qiao Xiang** received the bachelor's degree in information security and the bachelor's degree in economics from Nankai University in 2007, and the master's and Ph.D. degrees in computer science from Wayne State University in 2012 and 2014, respectively. He is a faculty member with Xiamen University. He was previously an Associate Research Scientist with the Department of Computer Science, Yale University. His research interests include software-defined networking, resource discovery and orchestration in collaborative data sciences, interdomain routing, and wireless cyber-physical systems. From 2016 to 2016, he was a Post-Doctoral Fellow with the Department of Computer Science, Yale University. From 2014 to 2015, he was a Post-Doctoral Fellow with the School of Computer Science, McGill University.



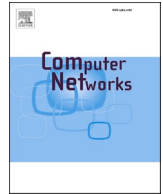
**Guang Cheng** received the Ph.D. degree in computer science from Southeast University, Nanjing, China, in 2003. From 2006 to 2007, he held a post-doctoral position with the School of Electrical and Computer Engineering, Georgia Institute of Technology. He is currently a Full Professor with the School of Cyber Science and Engineering, Southeast University. His research interests include network security, network measurement, and traffic sampling in computer network.

**Update**

**Computer Networks**

Volume 237, Issue , December 2023, Page

DOI: <https://doi.org/10.1016/j.comnet.2023.110098>



## Corrigendum

## Corrigendum to “FastTraffic: A lightweight method for encrypted traffic fast classification” [Computer Networks, Volume 235, November 2023, 109965]

Yuwei Xu<sup>a,b,c,\*</sup>, Jie Cao<sup>a,c</sup>, Kehui Song<sup>c,d</sup>, Qiao Xiang<sup>e</sup>, Guang Cheng<sup>a,b,c</sup>

<sup>a</sup> School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, China

<sup>b</sup> Purple Mountain Laboratories, Nanjing, 211111, China

<sup>c</sup> Jiangsu Province Engineering Research Center of Security for Ubiquitous Network, Nanjing, 211189, China

<sup>d</sup> College of Computer Science, NanKai University, Tianjin, 300071, China

<sup>e</sup> School of Informatics, Xiamen University, Xiamen, 361005, China

The authors regret that the results of the experiments in Table 7 were not correctly checked during proofing. The authors would like to apologize for any inconvenience caused.

In Table 7, ET-BERT's metrics were corrected for experimental results in T1 and T2, so Table 7 should be corrected to:

DOI of original article: <https://doi.org/10.1016/j.comnet.2023.109965>.

\* Corresponding author at: School of Cyber Science and Engineering, Southeast University, Nanjing, 211189, China.

E-mail address: [xuyw@seu.edu.cn](mailto:xuyw@seu.edu.cn) (Y. Xu).

<https://doi.org/10.1016/j.comnet.2023.110098>



**Table 7**

Classification performance comparison among nine ETC methods on T1 and T2. Results are in the format Avg.( $\pm$ Std.) obtained over 10-fold.

| Task               | T1 [%]                             |                                    |                                    |                                    | T2 [%]                             |                                    |                                    |                                    |
|--------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|------------------------------------|
| Method             | AC                                 | PR                                 | RC                                 | F1                                 | AC                                 | PR                                 | RC                                 | F1                                 |
| <b>1DCNN</b>       | 91.52( $\pm$ 0.13)                 | 93.12( $\pm$ 0.15)                 | 93.11( $\pm$ 0.22)                 | 93.09( $\pm$ 0.18)                 | 91.78( $\pm$ 1.12)                 | 86.08( $\pm$ 1.63)                 | 79.47( $\pm$ 1.57)                 | 81.83( $\pm$ 1.82)                 |
| <b>TSCRNN</b>      | 88.04( $\pm$ 1.02)                 | 89.36( $\pm$ 0.97)                 | 89.41( $\pm$ 1.54)                 | 89.26( $\pm$ 1.22)                 | 80.25( $\pm$ 1.00)                 | 77.59( $\pm$ 1.73)                 | 76.11( $\pm$ 1.56)                 | 76.21( $\pm$ 1.48)                 |
| <b>LSTM-Att</b>    | 88.01( $\pm$ 1.03)                 | 89.38( $\pm$ 0.97)                 | 89.03( $\pm$ 1.10)                 | 88.92( $\pm$ 1.11)                 | 80.54( $\pm$ 1.02)                 | 79.16( $\pm$ 2.20)                 | 74.55( $\pm$ 1.10)                 | 76.05( $\pm$ 0.70)                 |
| <b>DeepPacket</b>  | 90.66( $\pm$ 1.19)                 | 91.18( $\pm$ 1.37)                 | 89.50( $\pm$ 1.31)                 | 89.83( $\pm$ 1.33)                 | 89.05( $\pm$ 0.32)                 | 88.67( $\pm$ 0.53)                 | 86.68( $\pm$ 0.89)                 | 87.23( $\pm$ 0.73)                 |
| <b>SAM</b>         | 91.59( $\pm$ 0.17)                 | 92.46( $\pm$ 0.35)                 | 91.99( $\pm$ 0.18)                 | 92.00( $\pm$ 0.17)                 | 91.02( $\pm$ 0.22)                 | 93.56( $\pm$ 0.60)                 | 92.27( $\pm$ 0.70)                 | 92.72( $\pm$ 0.58)                 |
| <b>ET-BERT</b>     | 97.70( $\pm$ 0.92)                 | 97.55( $\pm$ 0.85)                 | 97.50( $\pm$ 0.80)                 | 97.51( $\pm$ 0.83)                 | 98.43( $\pm$ 1.07)                 | 98.41( $\pm$ 1.12)                 | 98.46( $\pm$ 1.08)                 | 98.43( $\pm$ 1.10)                 |
| <b>MATEC</b>       | 73.20( $\pm$ 0.72)                 | 84.43( $\pm$ 1.04)                 | 82.40( $\pm$ 1.03)                 | 82.87( $\pm$ 0.81)                 | 69.21( $\pm$ 0.69)                 | 73.32( $\pm$ 1.47)                 | 65.40( $\pm$ 2.03)                 | 68.24( $\pm$ 1.43)                 |
| <b>Datanet</b>     | 91.85( $\pm$ 0.31)                 | 91.57( $\pm$ 0.40)                 | 91.06( $\pm$ 0.30)                 | 91.17( $\pm$ 0.32)                 | 87.19( $\pm$ 0.44)                 | 84.43( $\pm$ 1.09)                 | 82.85( $\pm$ 0.95)                 | 83.16( $\pm$ 0.75)                 |
| <b>FastTraffic</b> | <b>94.50(<math>\pm</math>0.13)</b> | <b>94.77(<math>\pm</math>0.23)</b> | <b>94.26(<math>\pm</math>0.08)</b> | <b>94.40(<math>\pm</math>0.12)</b> | <b>92.24(<math>\pm</math>0.54)</b> | <b>93.58(<math>\pm</math>0.33)</b> | <b>92.84(<math>\pm</math>0.49)</b> | <b>93.12(<math>\pm</math>0.35)</b> |